

Graph Traversal and Genome Assembly

Eulerian vs. Hamiltonian Paths, Hierholzer's Algorithm, De Bruijn Graphs

Sergio Peignier

Outline

1. Eulerian Graphs
2. Hamiltonian Graphs
3. Comparison: Eulerian vs. Hamiltonian
4. Hierholzer's Algorithm
5. Modern DFS Stack Implementation
6. De Bruijn Graphs in Genome Assembly
7. Real-World Limitations & Adaptations

1. Eulerian Paths and Circuits: Definitions

Eulerian Path

A path in a finite graph that visits **every edge exactly once**. Vertices may be revisited multiple times.

Eulerian Circuit (or Cycle)

An Eulerian path that starts and ends at the **same vertex**.

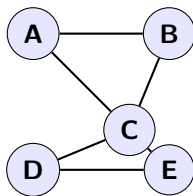
Undirected Graphs Existence Conditions

- **Eulerian Circuit:** Graph is connected and every vertex has an **even degree**.
- **Eulerian Path:** Graph is connected and *exactly zero or two* vertices have an **odd degree**.

Directed Graphs Existence Conditions

- **Eulerian Circuit:** Strongly connected, and $\text{in-degree}(v) = \text{out-degree}(v)$ for all v .
- **Eulerian Path:** Strongly connected, with at most one node having $\text{out} - \text{in} = 1$ (start) and one having $\text{in} - \text{out} = 1$ (end).

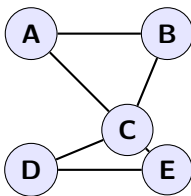
1. Eulerian Graphs: Example



Bow-Tie Graph Analysis:

- Vertices A, B, D, E have **degree 2**.
- Vertex C has **degree 4**.
- **Result:** All vertices have even degree $\rightarrow$ Contains an Eulerian Circuit

1. Eulerian Graphs: Example



Bow-Tie Graph Analysis:

- Vertices A, B, D, E have **degree 2**.
- Vertex C has **degree 4**.
- **Result:** All vertices have even degree $\rightarrow$ Contains an Eulerian Circuit

Eulerian Circuit Sequence

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow C \rightarrow A$

All 6 edges traversed once; Node C visited twice.

2. Hamiltonian Paths and Cycles: Definitions

Hamiltonian Path

A continuous path in a graph that visits **every vertex (node) exactly once**.

Hamiltonian Cycle

A closed loop that visits every vertex exactly once and returns to the starting vertex.

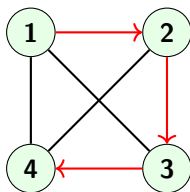
Dirac's Theorem

If every vertex in a simple graph with $n \geq 3$ vertices has a degree of at least $n/2$, the graph contains a Hamiltonian cycle (and thus a Hamiltonian path).

Key Distinction

Unlike Eulerian paths (which require visiting all *edges*), a Hamiltonian path does **not** need to traverse every edge in the graph. Edges can be left unused.

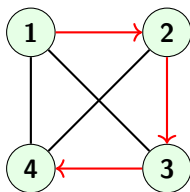
2. Hamiltonian Graphs: Example



Complete Graph K_4 Analysis:

- Total Vertices: 4
- Total Edges: 6

2. Hamiltonian Graphs: Example



Complete Graph K_4 Analysis:

- Total Vertices: 4
- Total Edges: 6

Valid Hamiltonian Path

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$$

Visits all 4 vertices once. Leaves edges (1,3), (2,4), and (1,4) uncrossed.

3. Eulerian vs. Hamiltonian: Structural Differences

Feature	Eulerian Path	Hamiltonian Path
Target Domain	Edges	Vertices
Rule	Traverse every edge once	Visit every vertex once
Node Repetition	Allowed	Strictly Forbidden
Unused Edges	Forbidden	Allowed
Verification Test	Simple degree condition	No simple necessary/sufficient test
Complexity	$\mathcal{O}(E)$ (Polynomial / Linear)	NP-Complete ($\mathcal{O}(2^n)$ via Dynamic Prog.)

Computational Significance

Eulerian problems are computationally easy (P-class), whereas Hamiltonian problems are computationally hard (NP-complete).

4. Hierholzer's Algorithm: Core Principle

Concept (Carl Hierholzer, 1873)

Hierholzer's algorithm constructs an Eulerian circuit in $O(|E|)$ time by iteratively building sub-cycles and splicing them together into a master traversal.

Algorithmic Strategy:

- ① Start at any vertex v and follow unvisited outgoing edges until returning to v (forms an initial sub-cycle C_1).
- ② If there are remaining unvisited edges connected to C_1 :
 - Pick a vertex $u \in C_1$ with unvisited outgoing edges.
 - Construct a second sub-cycle C_2 starting and ending at u .
 - **Splice** C_2 into C_1 at vertex u .
- ③ Repeat until all edges are exhausted.

5. Modern DFS Stack Implementation

Instead of manually splicing sub-cycles, modern implementations use a **Post-Order DFS with a Stack**.

Algorithm Mechanics:

- ① Push starting node onto a Stack.
- ② While Stack is non-empty:
 - If top node has unvisited edges: pop an edge, push neighbor to Stack.
 - If top node has **no unvisited edges**: pop node and append to Circuit.
- ③ Reverse Circuit to get final path.

Advantages & Complexity:

- **Implicit Splicing:** Post-order traversal automatically merges sub-cycles at junction nodes.
- **Time Complexity:** $\mathcal{O}(|E| + |V|)$ — every edge is processed and popped once.
- **Space Complexity:** $\mathcal{O}(|E| + |V|)$ for stack and adjacency list storage.

6. De Bruijn Graphs: Principles and Construction

Used to assemble long biological sequences from short sequencing reads.

Construction Rules for Parameter k

- **Nodes:** Substrings of length $k - 1$ (called $(k - 1)$ -mers).
- **Directed Edges:** Observed substrings of length k (k -mers). An edge connects its prefix of length $k - 1$ to its suffix of length $k - 1$.

Example Reads ($k = 4$):

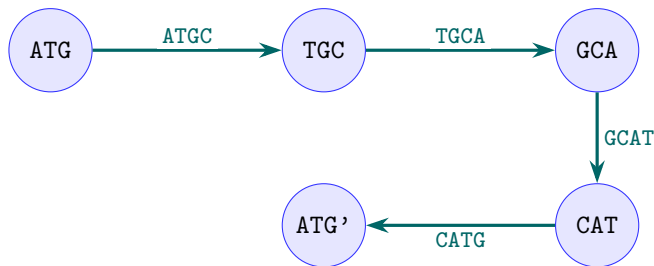
- Read 1: ATGCA $\rightarrow$ 4-mers: ATGC, TGCA
- Read 2: TGCAT $\rightarrow$ 4-mers: TGCA, GCAT

Graph Mapping:

- ATG $\xrightarrow{\text{ATGC}}$ TGC
- TGC $\xrightarrow{\text{TGCA}}$ GCA
- GCA $\xrightarrow{\text{GCAT}}$ CAT

6. Visualizing De Bruijn Graph Construction

Longer Read Sequence: ATGCA + TGCAT + GCATG → Reconstructed Target: ATGCATG



Traversing the Path

$$\text{ATG} \xrightarrow{\text{ATGC}} \text{TGC} \xrightarrow{\text{TGCA}} \text{GCA} \xrightarrow{\text{GCAT}} \text{CAT} \xrightarrow{\text{CATG}} \text{ATG}$$

Spelling the node prefixes along the Eulerian path reconstructs the full sequence: ATGCATG

6. De Bruijn Graphs vs. Overlap Graphs

Overlap-Layout-Consensus (OLC)

- **Nodes:** Full Reads
- **Edges:** Overlaps between reads
- **Traversal Goal:** Find a **Hamiltonian Path**
- **Complexity:** **NP-Hard** (Intractable for millions of short reads)

De Bruijn Graph Assemblers

- **Nodes:** $(k - 1)$ -mers
- **Edges:** k -mers
- **Traversal Goal:** Find an **Eulerian Path**
- **Complexity:** **Linear** $O(|E|)$ using Hierholzer's algorithm

7. Real-World Artifacts in De Bruijn Graphs

Real sequencing data breaks ideal Eulerian assumptions due to base errors and repeats.

Artifact	Biological Cause	Graph Topology
Tips (Dead ends)	Sequencing errors near read ends	Short dead-end branches
Bubbles	Single-base errors or SNPs	Paths split and quickly re-converge
Tangled Loops	Genomic repeats longer than k	Complex cycles with high node degrees
Chimeric Edges	Errors / Low abundance k -mers	Disconnected / low-coverage edges

Impact

In-degree and Out-degree balances are destroyed, making pure Eulerian path search impossible without pre-cleaning.

7. Algorithmic Adaptations

1. Tip Trimming and Bubble Popping

- **Tip Trimming:** Remove short dead-end branches below a length threshold.
- **Bubble Popping:** Align parallel paths in a bubble; merge them into a single consensus path based on coverage depth.

2. k -mer Abundance Filtering

Remove low-frequency k -mers (e.g., coverage count = 1) prior to graph creation.

3. Unitig Compaction

Collapse non-branching simple chains ($v_{in} = 1, v_{out} = 1$) into single super-edges called **unitigs** to reduce graph memory.

4. Repeat Resolution

Use **paired-end read mappings** to resolve high-degree repeat nodes, guiding Hierholzer's traversal through ambiguous loops.

Summary

- **Eulerian vs. Hamiltonian:** Eulerian searches focus on edges ($\mathcal{O}(|E|)$), while Hamiltonian searches focus on vertices (NP-complete).
- **Hierholzer's Algorithm:** Solves the Eulerian path problem in linear time using a post-order DFS stack.
- **De Bruijn Graphs:** Transforming genome assembly from an OLC (Hamiltonian) problem to a k -mer (Eulerian) problem revolutionized modern bioinformatics.
- **Real-World Assembly:** Requires pre-processing heuristics (tip trimming, bubble popping, unitig compaction) to handle biological noise before running Eulerian traversals.